

TITULO:

REPOSITORIO GIT



Implantación de Aplicaciones Web

Índice

Repositorio Git.....	3
1. Inicialización del repositorio.....	3
2. Primeros Archivos.....	4
3. Comprueba el estado del repositorio antes de añadir nada.....	4
4. Primer commit.....	5
5. Ignorar archivos.....	6
6. Modificación y comparación.....	7
7. Segundo commit.....	7
8. Renombrar un archivo.....	8
9. Eliminar un archivo.....	9
10. Un commit erróneo.....	9

Repositorio Git

En esta práctica vamos a configurar nuestro primer repositorio **Git**, para ello deberás realizar los siguientes ejercicios.

1. Inicialización del repositorio

Crea una carpeta llamada **tunombre_tusapellidos** y conviértela en un repositorio Git.

Para ello crearemos la carpeta y usamos `git init` para crear la carpeta oculta `.git`, que contiene el historial y la configuración del repositorio, no debemos modificar `.git` manualmente.

```
sergio@Sergio-PC:~$ mkdir sergio_mesa_mejias
sergio@Sergio-PC:~$ git init sergio_mesa_mejias
hint: Usando 'master' como el nombre de la rama inicial. Este nombre de rama pre
determinado
hint: está sujeto a cambios. Para configurar el nombre de la rama inicial para u
sar en todos
hint: de sus nuevos repositorios, reprimiendo esta advertencia, llama a:
hint:
hint:   git config --global init.defaultBranch <nombre>
hint:
hint: Los nombres comúnmente elegidos en lugar de 'master' son 'main', 'trunk' y
hint: 'development'. Se puede cambiar el nombre de la rama recién creada mediant
e este comando:
hint:
hint:   git branch -m <nombre>
Inicializado repositorio Git vacío en /home/sergio/sergio_mesa_mejias/.git/
sergio@Sergio-PC:~$ cd sergio_mesa_mejias/
sergio@Sergio-PC:~/sergio_mesa_mejias$
```

2. Primeros Archivos

Dentro de la carpeta, crea los siguientes archivos con un contenido mínimo (puede ser solo un título o comentario):

- **index.html**
- **estilos.css**
- **notas.txt** (con anotaciones personales de trabajo, que no deben formar parte del repositorio)

Creamos los ficheros con el siguiente contenido por ejemplo:

```
sergio@Sergio-PC:~/sergio_mesa_mejias$ cat index.html
<!doctype html>
<html>
<head><title>Mi sitio web</title></head>
<body><h1>Página principal</h1></body>
</html>
sergio@Sergio-PC:~/sergio_mesa_mejias$ cat estilos.css
/* Estilos iniciales */
body { font-family: sans-serif; }
sergio@Sergio-PC:~/sergio_mesa_mejias$ cat notas.txt
Anotaciones personales de trabajo
sergio@Sergio-PC:~/sergio_mesa_mejias$
```

3. Comprueba el estado del repositorio antes de añadir nada.

Los tres archivos aparecen como untracked porque Git los ve, pero todavía no los controla

```
sergio@Sergio-PC:~/sergio_mesa_mejias$ git status
En la rama master

No hay commits todavía

Archivos sin seguimiento:
(usa "git add <archivo>..." para incluirlo a lo que será confirmado)
estilos.css
index.html
notas.txt

no hay nada agregado al commit pero hay archivos sin seguimiento presentes (usa
"git add" para hacerles seguimiento)
```

4. Primer commit

Añade al área de preparación (*staging*) **index.html** y **estilos.css**, pero **no notas.txt**. Verifica con status que solo esos dos archivos están preparados.

Realiza el primer **commit** con un mensaje descriptivo, por ejemplo: "*Estructura inicial del sitio web*".

Añadiremos solamente los archivos que dicta el enunciado (index.html y estilos.css)

```
sergio@Sergio-PC:~/sergio_mesa_mejias$ git add index.html estilos.css
sergio@Sergio-PC:~/sergio_mesa_mejias$ git status
En la rama master

No hay commits todavía

Cambios a ser confirmados:
  (usa "git rm --cached <archivo>..." para sacar del área de stage)
    nuevos archivos: estilos.css
    nuevos archivos: index.html

Archivos sin seguimiento:
  (usa "git add <archivo>..." para incluirlo a lo que será confirmado)
    notas.txt

sergio@Sergio-PC:~/sergio_mesa_mejias$
```

El área de preparación, o "staging", es una zona intermedia donde seleccionamos exactamente qué cambios entrarán en el siguiente commit.

A continuación realizamos el commit:

```
sergio@Sergio-PC:~/sergio_mesa_mejias$ git commit -m "Estructura inicial del sitio web"
[master (commit-raíz) 9f1014e] Estructura inicial del sitio web
2 files changed, 7 insertions(+)
create mode 100644 estilos.css
create mode 100644 index.html
sergio@Sergio-PC:~/sergio_mesa_mejias$
```

5. Ignorar archivos

¿Porque consideras que puede ser necesario evitar que algunos archivos sean incluidos en nuestro repositorio? investiga sobre el fichero **.gitignore** y cómo tendrías que configurarlo para evitar tener que acordarte de excluir **notas.txt** manualmente cada vez.

Porque lo mas seguro es que tengamos archivos con notas personales, contraseñas, claves, configuraciones, etc que no queremos que se publique y además no tenemos que acordarnos de excluir esos archivos en cada commit gracias a .gitignore

Primero creamos el fichero .gitignore y ponemos esta linea para así evitar la publicacion de notas.txt en nuestros commits:

```
sergio@Sergio-PC:~/sergio_mesa_mejias$ cat .gitignore
notas.txt
```

Lo añadimos al repositorio y hacemos el commit correspondiente:

```
sergio@Sergio-PC:~/sergio_mesa_mejias$ git add .gitignore
sergio@Sergio-PC:~/sergio_mesa_mejias$ git commit -m "Configurar archivos ignorados"
[master 2273f38] Configurar archivos ignorados
 1 file changed, 1 insertion(+)
 create mode 100644 .gitignore
sergio@Sergio-PC:~/sergio_mesa_mejias$
```

Imaginemos que el archivo notas.txt se hubiese añadido anteriormente al repositorio antes de usar .gitignore, pues tendríamos que eliminar el archivo del repositorio con “git rm -cached notas.txt” y despues confirmar el cambio.

Ahora comprobamos con git status como el archivo notas.txt sigue existiendo en el ordenador pero ya no aparece como archivo pendiente:

```
sergio@Sergio-PC:~/sergio_mesa_mejias$ git status
En la rama master
nada para hacer commit, el árbol de trabajo está limpio
sergio@Sergio-PC:~/sergio_mesa_mejias$ ls -la
total 28
drwxrwxr-x   3 sergio sergio 4096 sep 22 12:25 .
drwx--x---+ 21 sergio sergio 4096 sep 22 12:09 ..
-rw-rw-r--   1 sergio sergio   58 sep 22 12:13 estilos.css
drwxrwxr-x   8 sergio sergio 4096 sep 22 12:30 .git
-rw-rw-r--   1 sergio sergio   10 sep 22 12:25 .gitignore
-rw-rw-r--   1 sergio sergio  112 sep 22 12:13 index.html
-rw-rw-r--   1 sergio sergio   34 sep 22 12:13 notas.txt
sergio@Sergio-PC:~/sergio_mesa_mejias$
```

6. Modificación y comparación

Edita **index.html** añadiendo una nueva sección (por ejemplo, un <footer>).
Antes de hacer commit, usa el comando adecuado para ver **exactamente qué líneas han cambiado** respecto a la última versión guardada.

Para comprobar esto tenemos que hacer un cambio en index.html y luego mirar que líneas han cambiado con la ayuda de "git diff"

```
sergio@Sergio-PC:~/sergio_mesa_mejias$ git diff
diff --git a/index.html b/index.html
index d02aa3b..e958e50 100644
--- a/index.html
+++ b/index.html
@@ -1,5 +1,8 @@
<!doctype html>
<html>
<head><title>Mi sitio web</title></head>
- <body><h1>Página principal</h1></body>
+ <body>
+ <h1>Página principal</h1>
+ <footer>© 2026 Sergio Mesa Mejías</footer>
+ </body>
</html>
sergio@Sergio-PC:~/sergio_mesa_mejias$
```

En un diff, las líneas que empiezan por "+" se han añadido y las que empiezan por "-" se han eliminado. "git diff" muestra cambios todavía no preparados.

7. Segundo commit

Guarda los cambios anteriores en un nuevo commit con mensaje: "Añadido footer a la página principal".

```
</html>
sergio@Sergio-PC:~/sergio_mesa_mejias$ git add index.html
git commit -m "Añadido footer a la página principal"
[master ff23dec] Añadido footer a la página principal
1 file changed, 4 insertions(+), 1 deletion(-)
sergio@Sergio-PC:~/sergio_mesa_mejias$
```

Si queremos ver el historial de commits

```
sergio@Sergio-PC:~/sergio_mesa_mejias$ git log --oneline
ff23dec (HEAD -> master) Añadido footer a la página principal
2273f38 Configurar archivos ignorados
9f1014e Estructura inicial del sitio web
sergio@Sergio-PC:~/sergio_mesa_mejias$
```

8. Renombrar un archivo

El cliente pide que el archivo de estilos se llame **styles.css** en lugar de **estilos.css**. Realiza el cambio de nombre de forma que Git lo registre correctamente (no valdría borrar y crear uno nuevo por separado). Confirma con status que Git detecta el renombrado. Haz commit del cambio.

Para ello usaremos git mv que tiene el mismo funcionamiento básico que el comando “mv”, y renombrar un archivo es muy simple, básicamente es renombrar y luego aceptar y publicar el cambio con un commit

```
sergio@Sergio-PC:~/sergio_mesa_mejias$ git mv estilos.css styles.css
fatal: mala fuente, fuente=estilos.css, destino=styles.css
sergio@Sergio-PC:~/sergio_mesa_mejias$ git status
En la rama master
Cambios a ser confirmados:
  (usa "git restore --staged <archivo>..." para sacar del área de stage)
    renombrados:    estilos.css -> styles.css

sergio@Sergio-PC:~/sergio_mesa_mejias$
```

Y hacemos el commit y comprobamos los cambios

```
sergio@Sergio-PC:~/sergio_mesa_mejias$ git commit -m "Renombrar estilos.css a styles.css"
[master f8f786d] Renombrar estilos.css a styles.css
1 file changed, 0 insertions(+), 0 deletions(-)
rename estilos.css => styles.css (100%)
sergio@Sergio-PC:~/sergio_mesa_mejias$ ls -la
total 28
drwxrwxr-x  3 sergio sergio 4096 sep 22 12:38 .
drwx--x---+ 21 sergio sergio 4096 sep 22 12:09 ..
drwxrwxr-x  8 sergio sergio 4096 sep 22 12:39 .git
-rw-rw-r--  1 sergio sergio  10 sep 22 12:25 .gitignore
-rw-rw-r--  1 sergio sergio 159 sep 22 12:32 index.html
-rw-rw-r--  1 sergio sergio  34 sep 22 12:13 notas.txt
-rw-rw-r--  1 sergio sergio  58 sep 22 12:13 styles.css
sergio@Sergio-PC:~/sergio_mesa_mejias$
```

Esto es preferible a borrar y crear archivos por separado porque expresa claramente la intención del cambio. Git puede detectar los archivos renombrados comparando contenidos, pero “git mv” deja la operación preparada de forma más concreta.

9. Eliminar un archivo

El archivo **index.html** ya no es necesario. Elimínalo tanto del sistema de archivos como del repositorio usando el comando de Git adecuado.

El comando git adecuado es “git rm”

```
sergio@Sergio-PC:~/sergio_mesa_mejias$ git rm index.html
rm 'index.html'
sergio@Sergio-PC:~/sergio_mesa_mejias$ git status
En la rama master
Cambios a ser confirmados:
  (usa "git restore --staged <archivo>..." para sacar del área de stage)
    borrados:        index.html

sergio@Sergio-PC:~/sergio_mesa_mejias$ git commit -m "Eliminar index.html"
[master 22f929d] Eliminar index.html
 1 file changed, 8 deletions(-)
 delete mode 100644 index.html
sergio@Sergio-PC:~/sergio_mesa_mejias$
```

Ya no existe el archivo:

```
sergio@Sergio-PC:~/sergio_mesa_mejias$ ls -la
total 24
drwxrwxr-x  3 sergio sergio 4096 sep 22 12:43 .
drwx--x---+ 21 sergio sergio 4096 sep 22 12:09 ..
drwxrwxr-x  8 sergio sergio 4096 sep 22 12:43 .git
-rw-rw-r--  1 sergio sergio  10 sep 22 12:25 .gitignore
-rw-rw-r--  1 sergio sergio  34 sep 22 12:13 notas.txt
-rw-rw-r--  1 sergio sergio  58 sep 22 12:13 styles.css
sergio@Sergio-PC:~/sergio_mesa_mejias$
```

10. Un commit erróneo

Por error hemos borrado **index.html** de forma incorrecta:

- Deshaz **solo el último commit**, ¿qué dos formas tendríamos para deshacerlo?

Una de las formas es “git revert HEAD” que cuando lo ejecutas crea un commit nuevo que invierte el anterior y te manda al editor de texto de un archivo en el que puedes cambiar el mensaje del commit, es decir que en nuestro caso recuperaríamos el index.html y podemos crear el commit correcto.

```
sergio@Sergio-PC: ~/sergio_mesa_mejias 104x55
GNU nano 8.4 /home/sergio/sergio_mesa_mejias/.git/COMMIT_EDITMSG
Revert "Recuperar index.html"

This reverts commit 22f929d4d85ae304bce77f0b42f9586055a6223c.

# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# En la rama master
# Cambios a ser confirmados:
#     nuevos archivos: index.html
#
```

Y aquí vuelve a estar nuestro index.html

```
sergio@Sergio-PC:~/sergio_mesa_mejias$ git revert HEAD
[master 44bd548] Revert "Recuperar index.html"
 1 file changed, 8 insertions(+)
 create mode 100644 index.html
sergio@Sergio-PC:~/sergio_mesa_mejias$ ls -la
total 28
drwxrwxr-x  3 sergio sergio 4096 sep 22 12:45 .
drwx--x---+ 21 sergio sergio 4096 sep 22 12:09 ..
drwxrwxr-x  8 sergio sergio 4096 sep 22 12:47 .git
-rw-rw-r--  1 sergio sergio   10 sep 22 12:25 .gitignore
-rw-rw-r--  1 sergio sergio  159 sep 22 12:45 index.html
-rw-rw-r--  1 sergio sergio   34 sep 22 12:13 notas.txt
-rw-rw-r--  1 sergio sergio   58 sep 22 12:13 styles.css
sergio@Sergio-PC:~/sergio_mesa_mejias$
```

HEAD` representa el commit actual.

Otra forma de revertir el ultimo commit seria “git reset --soft HEAD~1”

“HEAD~1” tambien significa el commit anterior. Con “--soft”, el commit desaparece del historial, pero los cambios quedan preparados en “staging”. Podemos corregirlos y crear otro commit.